Artificial Intelligence – Lab 5 (Group-A)

Implementation and Analysis of Markov Decision Processes

Robot Battery Management

Karanveer Gautam Sharma (2301MC10)

Prakhar Palod (2301MC17)

Prashant Nigam (2301MC18)

April 15, 2026

Contents

Abstract **3**

1 Introduction **4**

2 Mathematical Formulation **4**

2.1 Objective Function 4

2.2 Bellman Expectation Equation 4

2.3 Bellman Optimality Equation 4

3 Environment Description **5**

3.1 States 5

3.2 Actions 5

3.3 Transition Model 5

4 Task 1: Iterative Policy Evaluation **5**

4.1 Objective 5
4.2 Results 5
4.3 Observation 6

5 Task 2: Value Iteration **6**
5.1 Algorithm 6
5.2 Results 6
5.3 Insight 7

6 Task 3: Policy Iteration **7**
6.1 Steps 7
6.2 Results 7
6.3 Observation 8

7 Convergence Analysis **8**

8 Algorithm Comparison **8**

9 Insights **8**

10 Conclusion **9**

List of Figures

1 Expected Rewards by State 6
2 Value Iteration Convergence 7
3 Policy Iteration Results 8

List of Tables

1 State Transition Model 5
2 Baseline Policy Value 5
3 Optimal Policy 6
4 Comparison 8

Abstract

This report presents a comprehensive study of Markov Decision Processes (MDPs) applied to a robotic battery management problem. The system is modeled using states, actions, transition probabilities, and rewards.

We evaluate a baseline policy using Iterative Policy Evaluation and determine the optimal strategy using Value Iteration and Policy Iteration. Both methods converge to the same optimal policy, validating the correctness of Bellman-based dynamic programming.

The results demonstrate that long-term reward optimization may favor risk-taking strategies over conservative approaches.

1 Introduction

Markov Decision Processes (MDPs) provide a framework for sequential decision-making under uncertainty.

In this experiment, a robot must manage its battery efficiently while maximizing cumulative rewards. Each action leads to probabilistic state transitions, making optimal decision-making non-trivial.

The goal is to determine an optimal policy that maximizes expected long-term rewards.

2 Mathematical Formulation

An MDP is defined as (S, A, P, R, γ):

- S: States
- A: Actions
- $P(s'|s, a)$: Transition probability
- $R(s, a)$: Reward
- γ: Discount factor

2.1 Objective Function

$$V(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R_t\right]$$

2.2 Bellman Expectation Equation

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a)[R + \gamma V^\pi(s')]$$

2.3 Bellman Optimality Equation

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a)[R + \gamma V^*(s')]$$

3 Environment Description

3.1 States

High (0), Low (1), Charging (2)

3.2 Actions

Search (0), Wait (1)

3.3 Transition Model

State	Action	Next State	Probability	Reward
High	Search	High	0.6	+4
High	Search	Low	0.4	+4
Low	Search	High	0.4	-3
Low	Search	Low	0.6	+4
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

Table 1: State Transition Model

4 Task 1: Iterative Policy Evaluation

4.1 Objective

Evaluate baseline policy: [Search, Wait, Wait]

4.2 Results

Converged in 133 iterations.

State	Analytical	Empirical
High	18.108	18.085
Low	10.000	9.999
Charging	16.297	16.312

Table 2: Baseline Policy Value

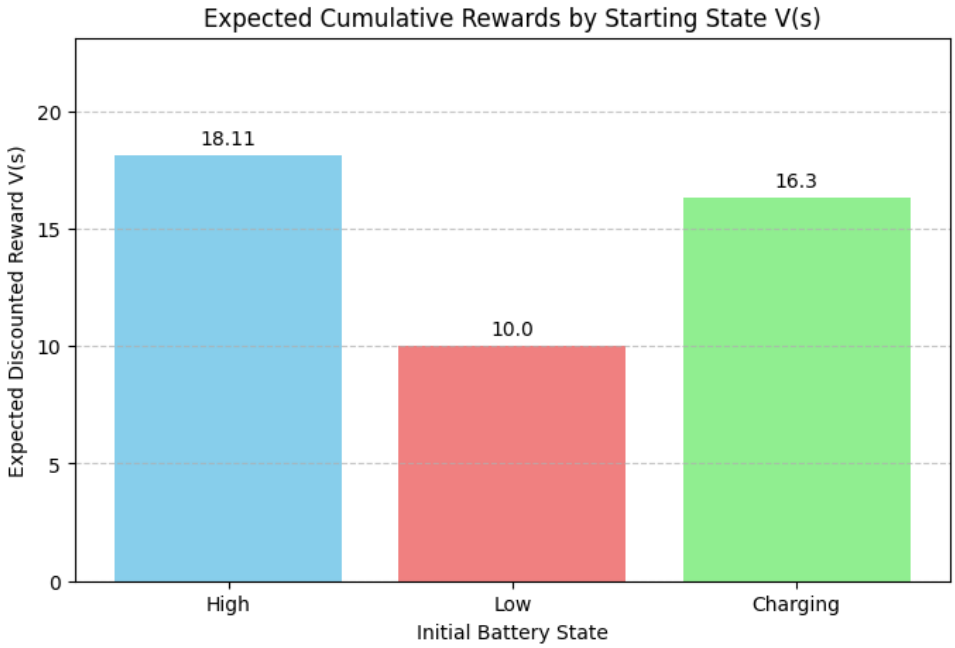


Figure 1: Expected Rewards by State

4.3 Observation

The policy is conservative and avoids risk, resulting in lower rewards.

5 Task 2: Value Iteration

5.1 Algorithm

- Initialize $V(s) = 0$
- Update using Bellman optimality
- Repeat until convergence

5.2 Results

Converged in 142 iterations.

State	$V^*(s)$	Policy
High	29.644	Search
Low	25.808	Search
Charging	26.679	Wait

Table 3: Optimal Policy

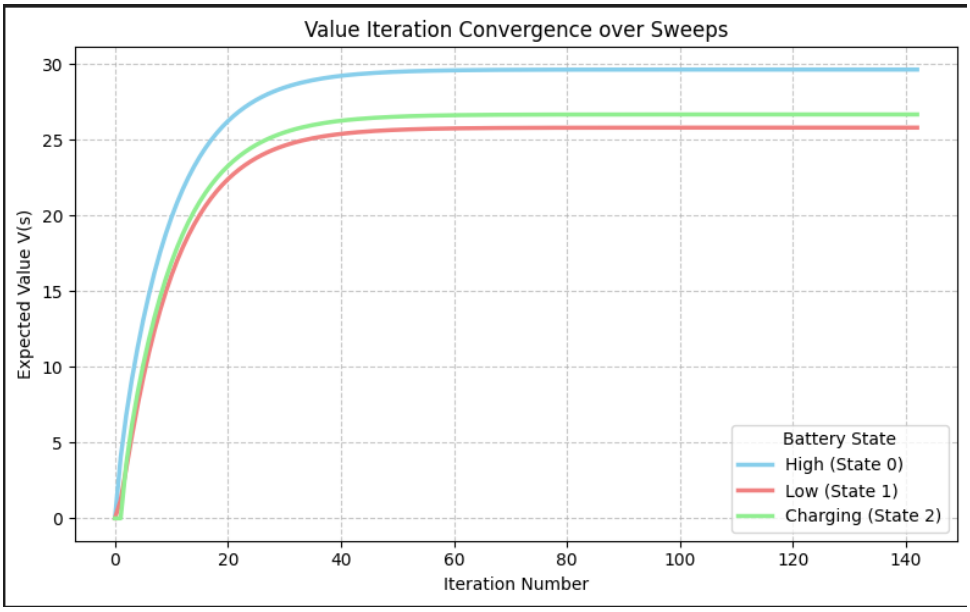


Figure 2: Value Iteration Convergence

5.3 Insight

Searching in Low state is optimal due to long-term reward dominance.

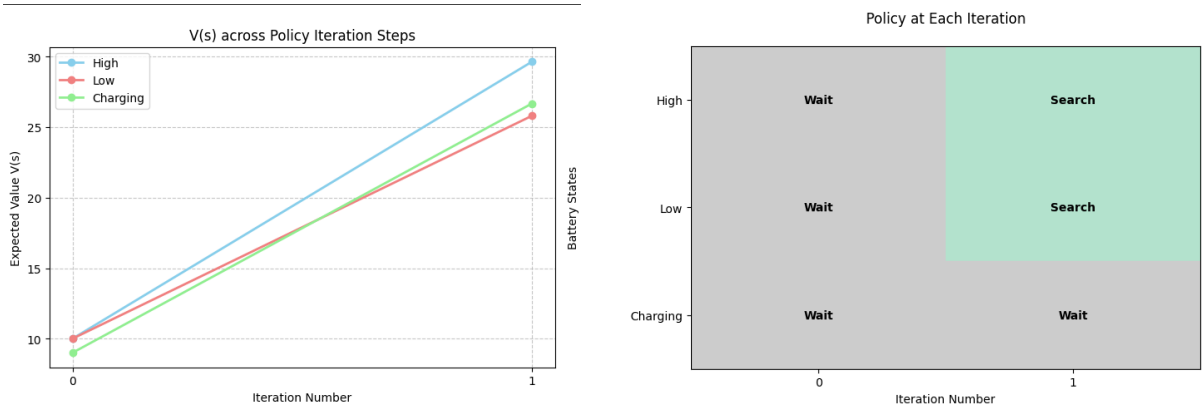
6 Task 3: Policy Iteration

6.1 Steps

- Policy Evaluation
- Policy Improvement

6.2 Results

- Iteration 0: $V = [10, 10, 9]$
- Iteration 1: $V = [29.644, 25.808, 26.679]$



(a) Value Function

(b) Policy Heatmap

Figure 3: Policy Iteration Results

6.3 Observation

Policy Iteration converges significantly faster.

7 Convergence Analysis

Both algorithms converge because Bellman operators are contraction mappings. Since $\gamma < 1$, repeated updates guarantee convergence to a unique solution.

8 Algorithm Comparison

Algorithm	Iterations	Speed
Value Iteration	142	Slow
Policy Iteration	1	Fast

Table 4: Comparison

9 Insights

The optimal strategy is:

$$\text{Search (High), Search (Low), Wait (Charging)}$$

This shows that taking calculated risks leads to better long-term outcomes.

10 Conclusion

The experiment demonstrates how MDPs can model real-world decision-making problems. Both Value Iteration and Policy Iteration converge to the same optimal policy, validating the approach.

The study highlights the importance of long-term planning over short-term safety.